ITBDA4-14 Project 2

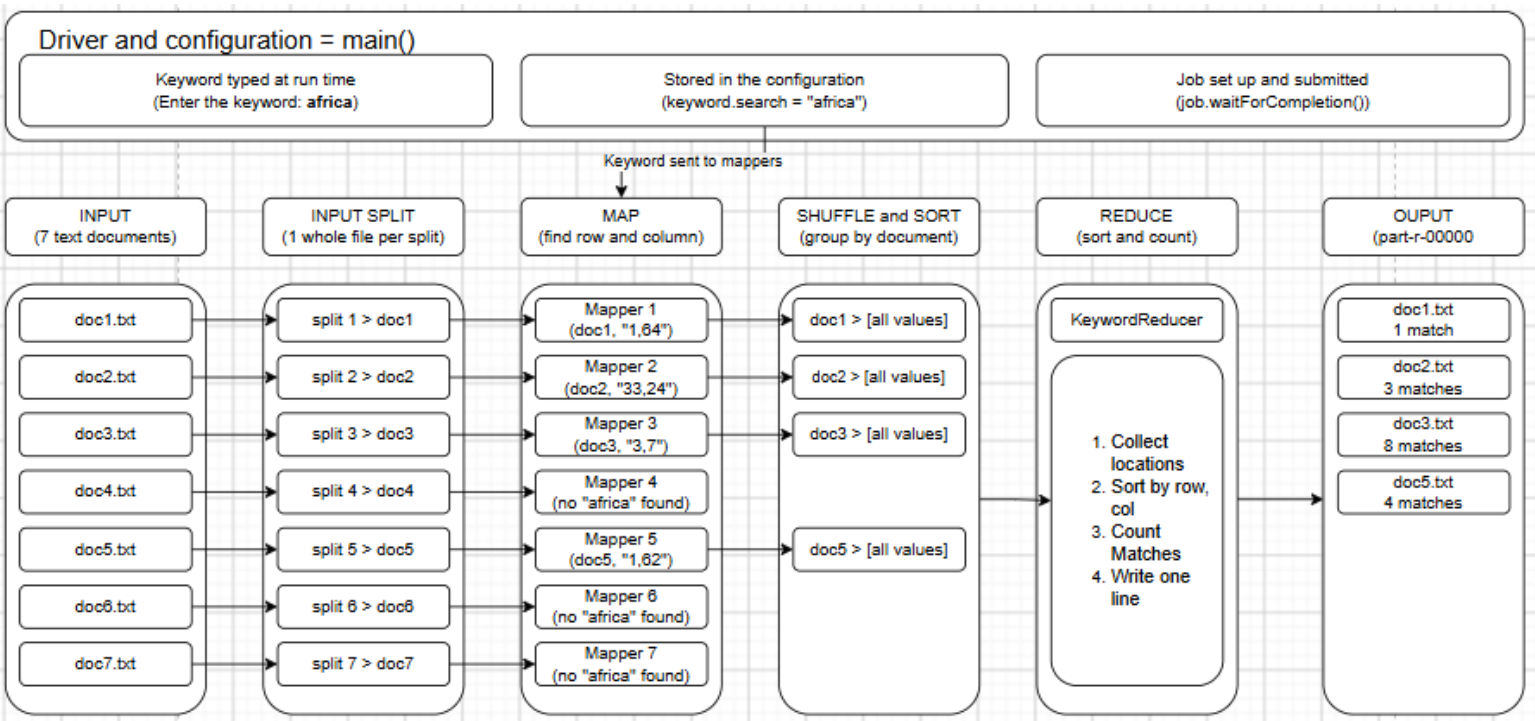
Andrew le Roux

EDUV12893867

Tyger valley Campus

Question 1

1.1) ***With the aid of a logical layout diagram, explain how you Will perform the keyword search operation into the MapReduce framework and yield the expected result.***



1.2) ***Provide a R or Java implementation of the Mapper for the above problem:***

```java
// 1.2 Provide a R or Java implementation of the Mapper for the above problem.
public static class KeywordMapper extends Mapper<LongWritable, Text, Text, Text> {

    private Pattern keywordPattern;
    private final Text documentName = new Text();
    private final Text location = new Text();
    private long lineNumber = 0;

    @Override
    protected void setup(Context context) throws IOException {

        String keyword = context.getConfiguration().get(KEYWORD_KEY);

        keywordPattern = Pattern.compile(
                "(?<![\\p{L}\\p{N}_])" + Pattern.quote(keyword) + "(?![\\p{L}\\p{N}_])",
                Pattern.CASE_INSENSITIVE);

        documentName.set(((FileSplit) context.getInputSplit()).getPath().getName());
    }

    @Override
    protected void map(LongWritable offset, Text value, Context context)
            throws IOException, InterruptedException {
        lineNumber++;

        Matcher m = keywordPattern.matcher(value.toString());
        while (m.find()) {
            int column = m.start() + 1;
            location.set(lineNumber + "," + column);
            context.write(documentName, location);
        }
    }
}
```

1.3) Provide a R or Java implementation of the Reducer class:

```java
// 1.3 Provide a R or Java implementation of the Reducer class
public static class KeywordReducer extends Reducer<Text, Text, Text, Text> {

    @Override
    protected void reduce(Text documentName, Iterable<Text> locations, Context context)
            throws IOException, InterruptedException {
        List<long[]> positions = new ArrayList<>();
        for (Text loc : locations) {
            String[] p = loc.toString().split(regex: ",");
            positions.add(new long[] { Long.parseLong(p[0]), Long.parseLong(p[1]) });
        }

        Collections.sort(positions, (a, b) -> a[0] != b[0]
                ? Long.compare(a[0], b[0]) : Long.compare(a[1], b[1]));

        StringBuilder sb = new StringBuilder();
        sb.append(positions.size()).append(positions.size() == 1 ? " match:" : " matches:");
        for (long[] p : positions) {
            sb.append(str: " (row ").append(p[0]).append(str: ", col ").append(p[1]).append(c: ')');
        }
        context.write(documentName, new Text(sb.toString()));
    }
}

public static class WholeFileTextInputFormat extends TextInputFormat {
    @Override
    protected boolean isSplitable(JobContext context, Path file) {
        return false;
    }
}
```

1.4: Provide an implementation of the Hadoop MapReduce configuration settings:

```
// 1.4 Provide an implementation of the Hadoop MapReduce configuration settings
Run | Debug
public static void main(String[] args) throws Exception {

    String input  = "input";
    String output = "output";

    System.out.print(s: "Enter the keyword: ");
    System.out.flush();
    String keyword = new BufferedReader(new InputStreamReader(System.in, StandardCharsets.UTF_8)).readLine();

    if (keyword == null || keyword.trim().isEmpty()) {
        System.err.println(x: "No keyword entered.");
        System.exit(status: 2);
    }
    keyword = keyword.trim();

    //Hadoop configuration settings
    Configuration conf = new Configuration();
    conf.set(KEYWORD_KEY, keyword);
    conf.set(name: "fs.defaultFS", value: "file:///");
    conf.set(name: "mapreduce.framework.name", value: "local");
    conf.set(name: "mapreduce.output.textoutputformat.separator", value: "\t");

    Job job = Job.getInstance(conf, "Keyword search: " + keyword);
    job.setJarByClass(cls: KeywordSearch.class);
    job.setMapperClass(cls: KeywordMapper.class);
    job.setReducerClass(cls: KeywordReducer.class);
    job.setNumReduceTasks(tasks: 1);
    job.setInputFormatClass(cls: WholeFileTextInputFormat.class);
    job.setOutputFormatClass(cls: TextOutputFormat.class);
    job.setMapOutputKeyClass(theClass: Text.class);
    job.setMapOutputValueClass(theClass: Text.class);
    job.setOutputKeyClass(theClass: Text.class);
```

```
job.setOutputValueClass(theClass: Text.class);

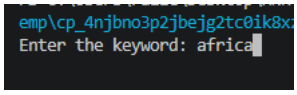
Path inPath = new Path(input);
Path outPath = new Path(output);
FileInputFormat.addInputPath(job, inPath);
FileOutputFormat.setOutputPath(job, outPath);

FileSystem fs = outPath.getFileSystem(conf);
if (fs.exists(outPath)) {
    fs.delete(outPath, recursive: true);
}

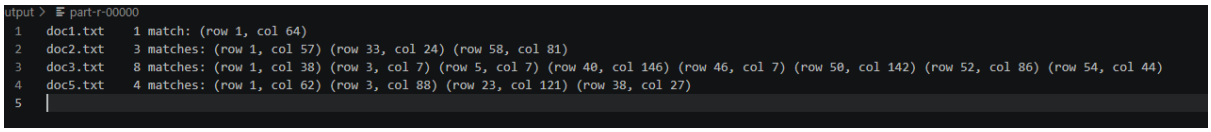
boolean ok = job.waitForCompletion(verbose: true);
if (!ok) {
    System.exit(status: 1);
}
```

Additional information regarding question 1:

After running script, you will be asked to enter keyword:



Click enter and the program will produce the result file under output “part-r-00000” which contains the following for the keyword “africa”:



(Codebase can be found in file “**KeywordSearch.java**”)

Question 2:

2.1) Given a collection of news files (news-zip), generate a data frame (COLI: file_name, coL2: content). Load the new content in each row and preprocess using appropriate data denoising techniques.

The 10 news files are provided are read into a data frame in r studios with one row per file. The file_name is in column 1 and the content in column 2. The denoising function is then run on the content to shrink the 3812 raw words into 2037 meaningful words. The final document-term matrix has 383 terms.

Denoising Techniques used:

1. Character normalisation – removed curly quotes, dashes and accented characters.
2. Lower casing – "Africa" and "africa" would have counted as different words.
3. Keep letters only – removed numbers, punctuation and symbols.
4. Remove English stop words – predefined noise_words array.
5. Lemmatisation – Generalised different form of one word ("providers" become "provider")
6. Remove filler words – Month and day names, source tag (sanews) and reporting verbs such as "say".
7. Remove 1-2 letter tokens and extra spaces
8. Drop rare and very common terms (in 2.2) – removed words that were only used once and words in 60% or more of files.

Code for 2.1:

```
#  2.1  DATA FRAME AND PREPROCESSING

files <- list.files(news_dir, pattern = "\\.txt$", full.names = TRUE)
files <- files[order(as.numeric(str_extract(basename(files), "\\d+")))]

news_df <- data.frame(
  file_name = basename(files),
  content   = sapply(files, function(f) paste(readLines(f, encoding = "UTF-8", warn = FALSE), collapse = " ")),
  row.names = NULL
)

noise_words <- c("monday", "tuesday", "wednesday", "thursday", "friday", "saturday",
                 "sunday", "january", "february", "march", "april", "may", "june",
                 "july", "august", "september", "october", "november", "december",
                 "sanews", "gov", "say", "said", "also", "will", "can", "one", "two",
                 "per", "new", "must", "well", "year", "mr", "ms", "minister", "make",
                 "need", "ensure", "include", "take", "however", "many", "much",
                 "first", "last", "time", "like", "get", "just", "come", "part")

denoise <- function(text) {
  text <- iconv(text, from = "UTF-8", to = "ASCII//TRANSLIT", sub = " ")
  text <- tolower(text)
  text <- str_replace_all(text, "[^a-z ]", " ")
  text <- removeWords(text, stopwords("english"))
  text <- lemmatize_strings(text)
  text <- removeWords(text, c(stopwords("english"), noise_words))
  text <- str_replace_all(text, "\\b[a-z]{1,2}\\b", " ")
  str_squish(text)
}
news_df$content <- sapply(news_df$content, denoise, USE.NAMES = FALSE)
print(data.frame(file_name = news_df$file_name, content = substr(news_df$content, 1, 60)))
```

2.2) *using Latent Dirichlet Allocation (LOA), find the top n topics in the documents and display the list of top k words per topic including appropriate reporting Of word distribution per topic:*

I fitted the LDA with 6 topics (n_topics <- 6) and 10 words per topic (top_k <- 10) using Gibbs sampling with a fixed seed of 1234 so that the results repeat.

Each of the topics are a probability distribution over words (beta, P(word|topic)).

Each document is a mixture of topics (gamma, P(topic|document)).

Log-likelyhood per number of topics:

(k_check)

	topics	log_likelihood
1	2	-8033.308
2	3	-7736.848
3	4	-7561.018
4	5	-7458.375
5	6	-7373.932
6	7	-7302.151
7	8	-7231.557

Gains shrink quickly after 5 to 6 topics which indicates the elbow if placed in an elbow graph. I considered 6 as the most optimal number of topics, because 6 is the smallest number that will give distinct topics.

Top 10 words per topic with beta:

(word_table)

	rank	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6
1	1	africa (0.046)	priority (0.035)	cape (0.040)	government (0.082)	invoice (0.075)	eskom (0.045)
2	2	administration (0.031)	unemployment (0.028)	cold (0.040)	coalition (0.052)	pay (0.054)	police (0.041)
3	3	country (0.031)	meth (0.025)	weather (0.035)	role (0.025)	day (0.046)	sandf (0.023)
4	4	gxoyiya (0.031)	labour (0.021)	western (0.035)	national (0.021)	provider (0.038)	kidnapper (0.019)
5	5	official (0.031)	report (0.021)	expect (0.030)	policy (0.018)	payment (0.033)	member (0.019)
6	6	psc (0.027)	school (0.021)	week (0.025)	relationship (0.018)	national (0.029)	mine (0.019)
7	7	fund (0.023)	employment (0.018)	community (0.020)	finlay (0.015)	non (0.025)	power (0.019)
8	8	bribe (0.019)	good (0.018)	front (0.020)	general (0.015)	within (0.025)	team (0.019)
9	9	reform (0.019)	action (0.014)	report (0.020)	governance (0.015)	financial (0.021)	victim (0.019)
10	10	democratic (0.016)	build (0.014)	west (0.020)	party (0.015)	psc (0.021)	court (0.015)

Topic distribution per document with dominant topic:

	document	Topic 1	Topic 2	Topic 3	Topic 4	Topic 5	Topic 6	dominant_topic
1	news_1.txt	0.001	0.001	0.001	0.001	0.127	0.868	6
2	news_2.txt	0.020	0.967	0.010	0.001	0.001	0.001	2
3	news_3.txt	0.215	0.693	0.000	0.090	0.000	0.000	2
4	news_4.txt	0.001	0.001	0.107	0.001	0.001	0.888	6
5	news_5.txt	0.001	0.034	0.001	0.001	0.001	0.961	6
6	news_6.txt	0.001	0.001	0.001	0.001	0.997	0.001	5
7	news_7.txt	0.957	0.001	0.013	0.013	0.013	0.001	1
8	news_8.txt	0.000	0.008	0.000	0.987	0.000	0.004	4
9	news_9.txt	0.792	0.016	0.001	0.040	0.150	0.001	1
10	news_10.txt	0.001	0.001	0.919	0.036	0.001	0.042	3

Code for 2.2:

```
#  2.2  LDA

dtm <- DocumentTermMatrix(VCorpus(VectorSource(news_df$content)), control = list(wordLengths = c(3, Inf)))
m   <- as.matrix(dtm)
dtm <- dtm[, colSums(m) >= 2 & colSums(m > 0) < 0.6 * nrow(dtm)]
rownames(dtm) <- news_df$file_name

lda_control <- list(seed = 1234, burnin = 1000, iter = 2000, thin = 100, alpha = 0.1)

k_check <- data.frame(topics = 2:8)
k_check$log_likelihood <- sapply(k_check$topics, function(k)
  as.numeric(logLik(LDA(dtm, k = k, method = "Gibbs", control = lda_control))))
print(k_check)

lda_model   <- LDA(dtm, k = n_topics, method = "Gibbs", control = lda_control)
topic_words <- tidy(lda_model, matrix = "beta")
doc_topics  <- tidy(lda_model, matrix = "gamma")

top_terms <- topic_words %>% group_by(topic) %>%
  slice_max(beta, n = top_k, with_ties = FALSE) %>%
  mutate(rank = row_number()) %>% ungroup()

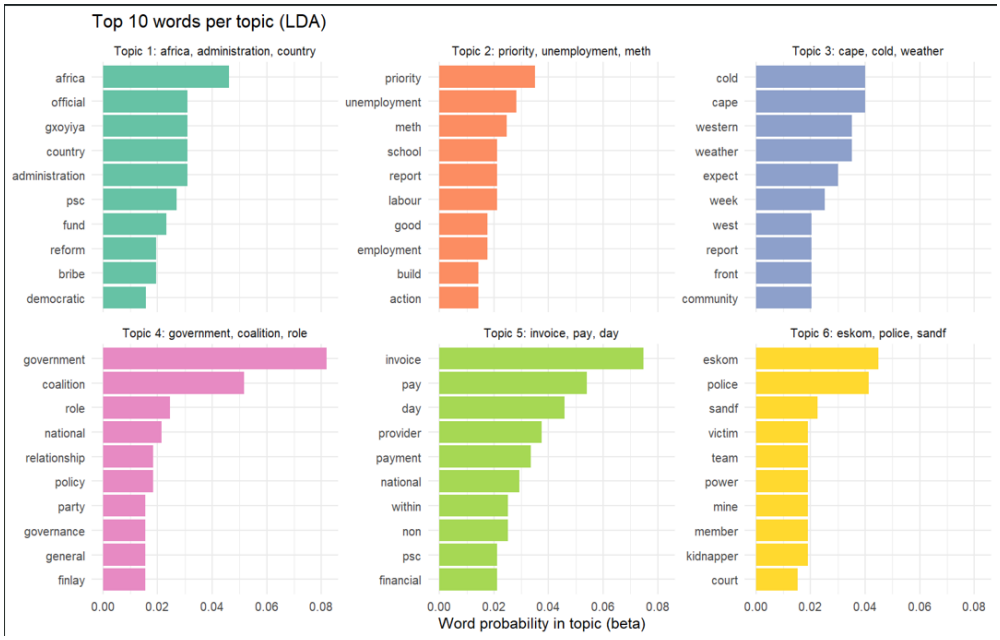
print(as.data.frame(top_terms %>%
                      mutate(cell = sprintf("%s (%.3f)", term, beta)) %>%
                      select(rank, topic, cell) %>%
                      pivot_wider(names_from = topic, values_from = cell, names_prefix = "Topic ")), row.names = FALSE)

print(top_terms %>% group_by(topic) %>% summarise(top_k_share = sum(beta)))

print(as.data.frame(doc_topics %>% mutate(gamma = round(gamma, 3)) %>%
                      pivot_wider(names_from = topic, values_from = gamma, names_prefix = "Topic ")), row.names = FALSE)
```

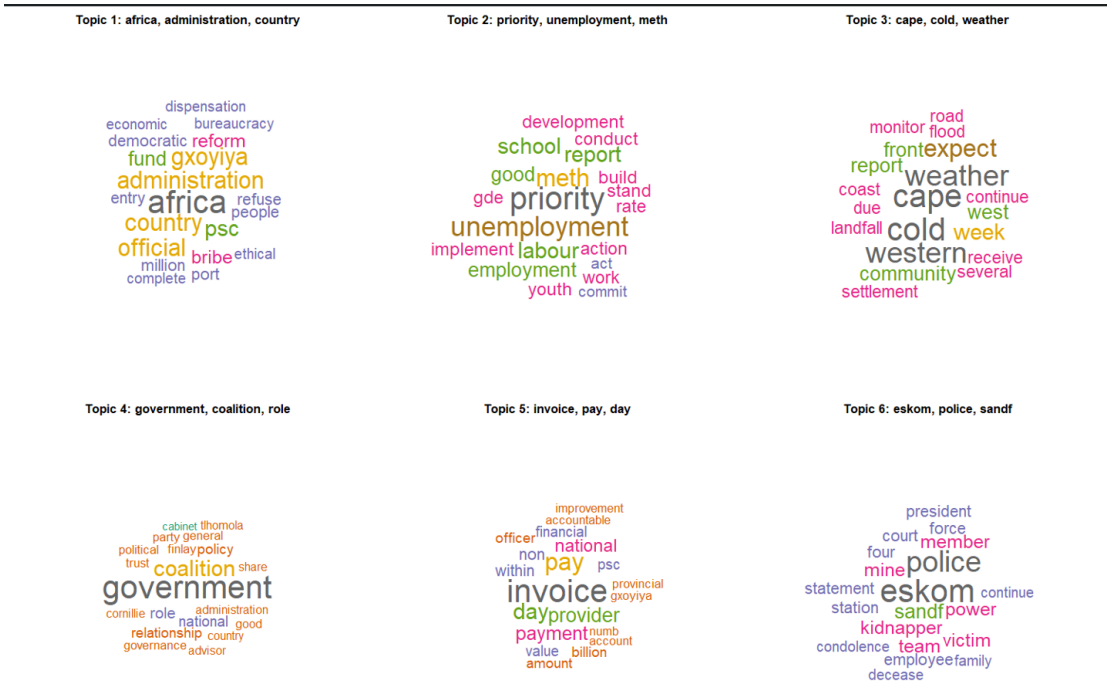
2.3) Provide appropriate Visualization for the top key words in each topic using at least two visualization methods:

Method 1: Bar chart of the top 10 words per topic where the bar length is the words probability in each topic:
(1_top_words_bar_chart)



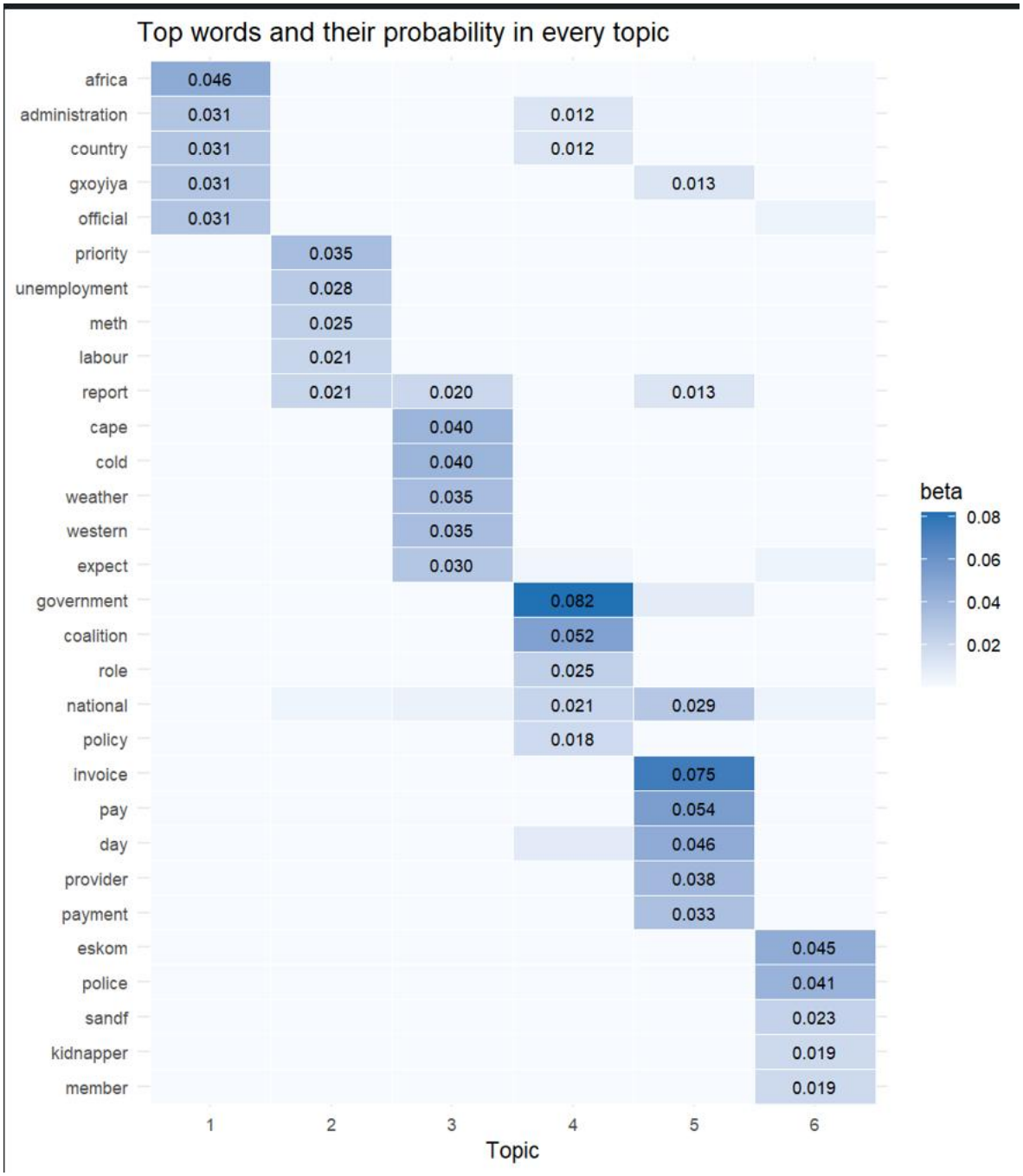
Method 2: Word cloud per topic where the word size is proportional to beta for the 20 most likely words in each topic:

(2_word_clouds)



Method 3: Heatmap of the top 5 words of all topics where each cell is a words probability in a topic:

(3_top_words_heatmap)



Code for 2.3:

```r
#  2.3  VISUALISATION

top_terms <- top_terms %>% group_by(topic) %>%
  mutate(label = paste0("Topic ", topic, ": ", paste(head(term, 3), collapse = ", "))) %>% ungroup()

# Method 1: bar chart
p1 <- ggplot(top_terms, aes(beta, reorder_within(term, beta, topic), fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ label, scales = "free_y") +
  scale_y_reordered() +
  scale_fill_brewer(palette = "Set2") +
  labs(title = paste("Top", top_k, "words per topic (LDA)"), x = "word probability in topic (beta)", y = NULL) +
  theme_minimal(base_size = 12)
print(p1)
ggsave("plots/1_top_words_bar_chart.png", p1, width = 11, height = 7, dpi = 150)

# Method 2: word clouds
png("plots/2_word_clouds.png", width = 1500, height = 1050, res = 130)
par(mfrow = c(2, 3), mar = c(0, 0, 2, 0))
for (t in 1:n_topics) {
  tw <- topic_words %>% filter(topic == t) %>% slice_max(beta, n = 20, with_ties = FALSE)
  wordcloud(tw$term, tw$beta, scale = c(3, 0.8), random.order = FALSE, rot.per = 0,
            colors = brewer.pal(8, "Dark2"))
  title(unique(top_terms$label[top_terms$topic == t]))
}
dev.off()

# Method 3: heatmap of the top 5 words of each topic
heat_order <- top_terms %>% filter(rank <= 5) %>% distinct(term, .keep_all = TRUE)
heat_data  <- topic_words %>% filter(term %in% heat_order$term) %>%
  mutate(term = factor(term, levels = rev(heat_order$term)))
p3 <- ggplot(heat_data, aes(factor(topic), term, fill = beta)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = ifelse(beta >= 0.01, sprintf("%.3f", beta), "")), size = 3) +
  scale_fill_gradient(low = "#f7fbff", high = "#2171b5") +
  labs(title = "Top words and their probability in every topic", x = "Topic", y = NULL) +
  theme_minimal(base_size = 11)
print(p3)
ggsave("plots/3_top_words_heatmap.png", p3, width = 7, height = 8, dpi = 150)
```

(Codebase can be found in file “**Question2_LDA.R**”)

Question 3:

3.1) Select a website of your own choosing with compliance of 'fair use of webscraping and extract the clean text web content from the site leading to the formation of an R data structure:

I selected Wikipedia to gather insights.

Four English Wikipedia articles were scraped:
Natural language processing:
(https://en.wikipedia.org/wiki/Natural_language_processing)

Large Language Model:
(https://en.wikipedia.org/wiki/Large_language_model)

Test Mining:
(https://en.wikipedia.org/wiki/Text_mining)

Automatic summarization:
(https://en.wikipedia.org/wiki/Automatic_summarization)

I chose Wikipedia because its content is open and rules allow automated reading of articles. To ensure fair use these methods were followed:

1. Respect robots.txt: “robotstxt::paths_allowed()” checks every link first and stops if any page is not allowed.
2. Identify the scraper: Wikipedia’s policy asks that there is a descriptive user agent with contact details
   (my_agent <- "Student text mining project (R rvest) - contact: andrew.ler23@gmail.com")
3. Do not overload the server: Between each request there is a 2 second pause.
4. Collect only what is needed: Only scrape and keep article text, no personal data is collected.

Rvest reads each page and selects the paragraphs (p) and headings (h2 and h3) with a xPath query. Any text inside navigation boxes, reference lists or tables are ignored. Every paragraph is labelled with the section it belongs to.

This results in a data frame of 232 paragraphs x 5 columns:

Column	Type	Content
Article	Chr	Article title
url	Chr	Web link
Section	Chr	Section heading
Paragraph_id	Int	Paragraph number
Raw_text	Chr	Paragraph text as scraped

Article	Paragraphs	Sections
Large language model	110	39
Automatic summarization	73	25
Text Mining	27	18
Natural Language processing	22	11

```
library(tidytext); library(textstem); library(ggplot2); library(igraph); library(ggraph)

urls <- c(
  "https://en.wikipedia.org/wiki/Natural_language_processing",
  "https://en.wikipedia.org/wiki/Large_language_model",
  "https://en.wikipedia.org/wiki/Text_mining",
  "https://en.wikipedia.org/wiki/Automatic_summarization"
)

my_agent <- "Student text mining project (R rvest) - contact: andrew.ler23@gmail.com"
dir.create("plots", showWarnings = FALSE)

#  3.1  FAIR USE WEB SCRAPING INTO AN R DATA STRUCTURE
allowed <- paths_allowed(urls)
print(data.frame(url = urls, allowed_by_robots_txt = allowed))
if (!all(allowed)) stop("robots.txt does not allow one of these pages - choose another page.")

skip_sections <- c("See also", "References", "Notes", "Further reading", "External links",
                   "Sources", "Citations", "Bibliography")

scrape_page <- function(url) {
  Sys.sleep(2)
  page  <- session(url, httr::user_agent(my_agent))
  title <- html_text2(html_element(page, "h1"))
  nodes <- html_elements(page, xpath = paste0(
    "//div[contains(@class,'mw-parser-output')]//*[self::p or self::h2 or self::h3]",
    "[not(ancestor::table)][not(ancestor::*[contains(@class,'navbox') or contains(@class,'reflist')])]"))

  section <- "Introduction"; skipping <- FALSE; rows <- list()
  for (node in nodes) {
    tag  <- html_name(node)
    text <- str_squish(html_text2(node))
    if (tag == "h2") {
      section  <- text
      skipping <- section %in% skip_sections
    } else if (tag == "h3") {
      if (!skipping) section <- text
    } else if (!skipping) {
      rows[[length(rows) + 1]] <- data.frame(section = section, raw_text = html_text2(node))
    }
  }
  data.frame(article = title, url = url, bind_rows(rows))
}

web_df <- bind_rows(lapply(urls, scrape_page)) %>%
  filter(str_squish(raw_text) != "") %>%
  mutate(paragraph_id = row_number(), .after = section)

str(web_df)
write.csv(web_df, "scraped_web_text.csv", row.names = FALSE)
```

3.2) Perform textual data preprocessing using all appropriate techniques that clean the data content of non-pertinent information for analysis.

The cleaning was done on paragraph level and word level. The paragraph level cleaning removes noise from Wikipedia and the web but keeps the sentences. The word level cleaning reduces the text to meaningful base words for the keyword statistics.

Level	Technique	Non-pertinent content removed
Paragraph	Remove citation markers	347 markers such as [12] and [citation needed]
Paragraph	Remove LaTeX math codes	{\displaystyle ...} formula code in 5 large language model paragraphs.
Paragraph	Remove empty brackets left behind by the steps above	Leftovers of page markup
Paragraph	Character normalisation	Curly quotes, dashes and accented characters
Paragraph	Drop fragments under 60 characters and duplicate paragraphs	Captions, single line notes and repeats
Word	Tokenise and lower case	“Model” and “model” were counted separately
Word	Keep letters only	Numbers and symbols
Word	Remove English stop words and filler words	“Stop_words”
Word	Lemmatise with domain corrections	Word forms: “models” become “model”. Correction: “datum” back to “data”, “mine” to “mining” ...
Word	Drop words under 3 letters	Leftover fragments

Before and after preprocessing:

Measure	Before	After
Paragraphs	232	225
Words	15376	7932 meaningful words
Unique words	-	2027

(Code for 3.2)

```
#  3.2  TEXT PREPROCESSING
web_df <- web_df %>%
  mutate(clean_text = raw_text %>%
           str_replace_all("\\[[^\\]]*\\]", " ") %>%
           str_replace_all("\\s*[\\\\{}]\\s*", " ") %>%
           str_replace_all("\\(\\s*\\)", " ") %>%
           iconv(from = "UTF-8", to = "ASCII//TRANSLIT", sub = " ") %>%
           str_squish()) %>%
  filter(str_length(clean_text) >= 60) %>%
  distinct(clean_text, .keep_all = TRUE)

custom_stop <- c("also", "use", "used", "using", "can", "may", "one", "two", "first",
                 "however", "e.g", "i.e", "etc", "include", "including", "example")
lemma_fix <- c(datum = "data", mine = "mining", llms = "llm")
words_df <- web_df %>%
  unnest_tokens(word, clean_text) %>%
  filter(!str_detect(word, "[^a-z]")) %>%
  anti_join(stop_words, by = "word") %>%
  mutate(word = lemmatize_words(word),
         word = ifelse(word %in% names(lemma_fix), lemma_fix[word], word)) %>%
  filter(!word %in% custom_stop, str_length(word) >= 3)

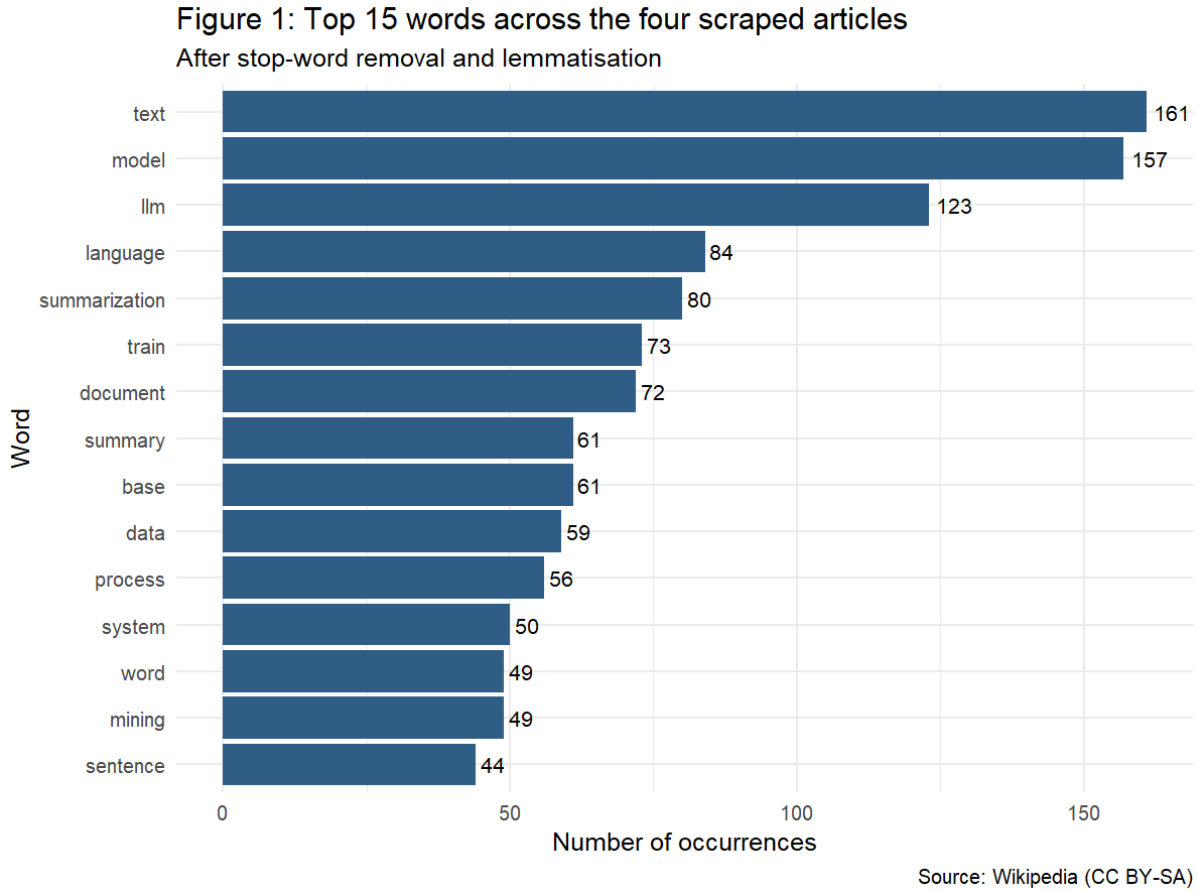
cat("\nAfter preprocessing:", nrow(web_df), "paragraphs,", nrow(words_df), "words,",
    n_distinct(words_df$word), "unique words\n")
```

3.3) Provide text summarisation of the collected information using at least two text summarisation techniques including at least two visualisations:

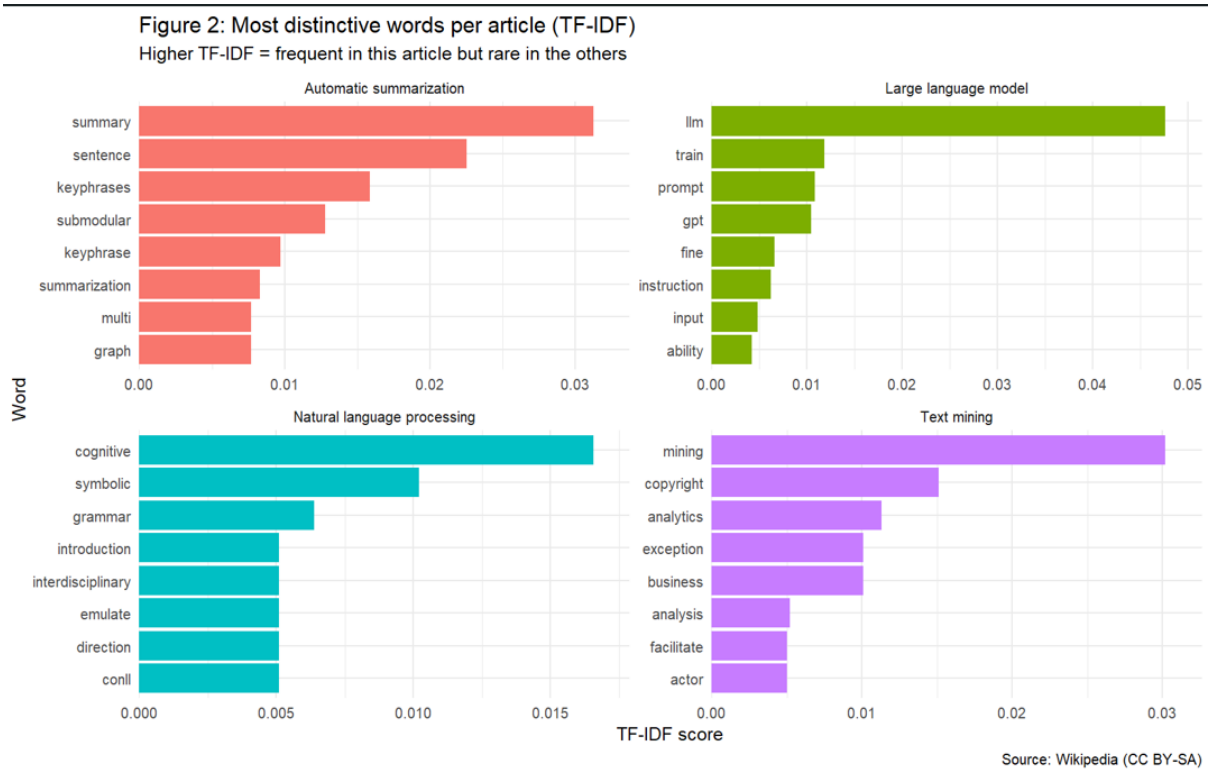
Summarisation techniques that were used:

	Technique	Type	What it produces
1	Keyword summary	Statistical, word level	Main topics
2	Frequency based extractive summary	Extractive, sentence level	The 4 sentences per article whose words are most frequent in that article
3	TextRank extractive summary	Extractive, graph-based	The 4 most central sentences per article

Technique 1: Keyword summary:



On the graph we can see that the articles are dominated by the words text (161 times) model (157 times) and llm (123 time). Then a bit less occurrences by language (84 times) and summarization (80 times). If put together, these words summarise the scraped content in one line. “Language models that process and summarize text.”

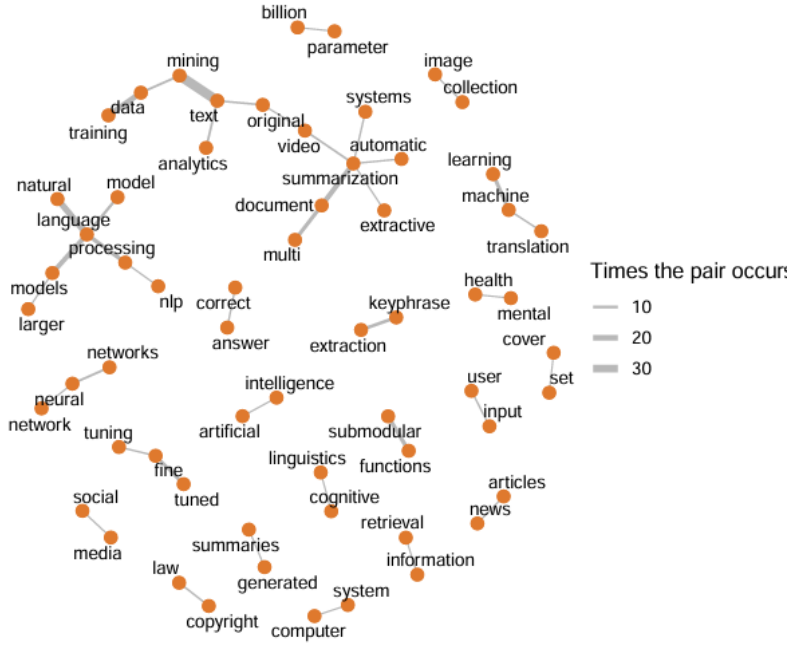


Here we can see what makes each article different. Using TF-IDF which scores a word highly when it is frequent in one article but rare in the others.

1. Automatic summarization explains how sentences and key phrases are selected to build summary
2. Large language model indicates the vocabulary that is used when building LLMs
3. Natural Language processing focuses on the history and theory of NLP.
4. Text mining covers business use and the legal side of mining text.

Figure 3: Network of the 40 most common word pairs (bigrams)

Linked words appear next to each other in the text



Source: Wikipedia (CC BY–SA)

This graph links words that appear next to each other, the thicker the line, the more frequent the pair appears.

We can see that the strongest pairs are text mining (Text and mining), natural language (Natural and language) and training data (training and data).

```
#Technique 1: keyword summary
top_words <- words_df %>% count(word, sort = TRUE) %>% slice_head(n = 15)

tfidf <- words_df %>% count(article, word) %>%
  bind_tf_idf(word, article, n) %>%
  group_by(article) %>% slice_max(tf_idf, n = 8, with_ties = FALSE) %>% ungroup()

bigrams <- web_df %>%
  unnest_tokens(bigram, clean_text, token = "ngrams", n = 2) %>%
  separate(bigram, c("w1", "w2"), sep = " ") %>%
  filter(!w1 %in% stop_words$word, !w2 %in% stop_words$word,
         !str_detect(w1, "[^a-z]"), !str_detect(w2, "[^a-z]")) %>%
  count(w1, w2, sort = TRUE)
```

```
f1 <- ggplot(top_words, aes(n, reorder(word, n))) +
  geom_col(fill = "#2f5d86") +
  geom_text(aes(label = n), hjust = -0.2, size = 3.5) +
  labs(title = "Figure 1: Top 15 words across the four scraped articles",
       subtitle = "After stop-word removal and lemmatisation",
       x = "Number of occurrences", y = "Word", caption = "Source: Wikipedia (CC BY-SA)") +
  theme_minimal(base_size = 12)
print(f1); ggsave("plots/1_top_words.png", f1, width = 8, height = 6, dpi = 150)

# Figure 2: words that characterise each article (TF-IDF)
f2 <- ggplot(tfidf, aes(tf_idf, reorder_within(word, tf_idf, article), fill = article)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ article, scales = "free") +
  scale_y_reordered() +
  scale_x_continuous(n.breaks = 4, expand = expansion(mult = c(0, 0.08))) +
  labs(title = "Figure 2: Most distinctive words per article (TF-IDF)",
       subtitle = "Higher TF-IDF = frequent in this article but rare in the others",
       x = "TF-IDF score", y = "Word", caption = "Source: Wikipedia (CC BY-SA)") +
  theme_minimal(base_size = 12)
print(f2); ggsave("plots/2_tfidf_per_article.png", f2, width = 11, height = 7, dpi = 150)

# Figure 3: network of the most common word pairs
set.seed(2026)
bigram_graph <- graph_from_data_frame(head(bigrams, 40))
f3 <- ggraph(bigram_graph, layout = "fr") +
  geom_edge_link(aes(edge_width = n), edge_colour = "grey60", alpha = 0.7) +
  geom_node_point(colour = "#e07a2e", size = 3) +
  geom_node_text(aes(label = name), repel = TRUE, size = 3.5) +
  scale_edge_width(range = c(0.4, 2.5), name = "Times the pair occurs") +
  labs(title = "Figure 3: Network of the 40 most common word pairs (bigrams)",
       subtitle = "Linked words appear next to each other in the text",
       caption = "Source: Wikipedia (CC BY-SA)") +
  theme_void(base_size = 12)
print(f3); ggsave("plots/3_bigram_network.png", f3, width = 10, height = 7, dpi = 150)
```

Technique 2 and 3: Sentence Summaries:

Both these techniques split the cleaned paragraphs into readable sentences. The script ignores sentences under 8 words and scores every sentence. The four highest scoring sentences are kept in their original order.

- Technique 2 (frequency-based (luhn): A sentence's score is calculated based on how frequent its content appears on an average across the article. Sentences that are built from the article's most common words rank the highest.

- Technique 3 (TextRank): Each sentence is seen as a node in a graph. When two sentences share words, they are linked. The link weight = shared words / (log of length 1 + log of length 2). PageRank is then used to give the highest score to sentences that are like many other sentences. This allows us to extract the main plot of the article.

Figure 4: TextRank importance of each sentence
The 4 highest-scoring sentences per article form the summary

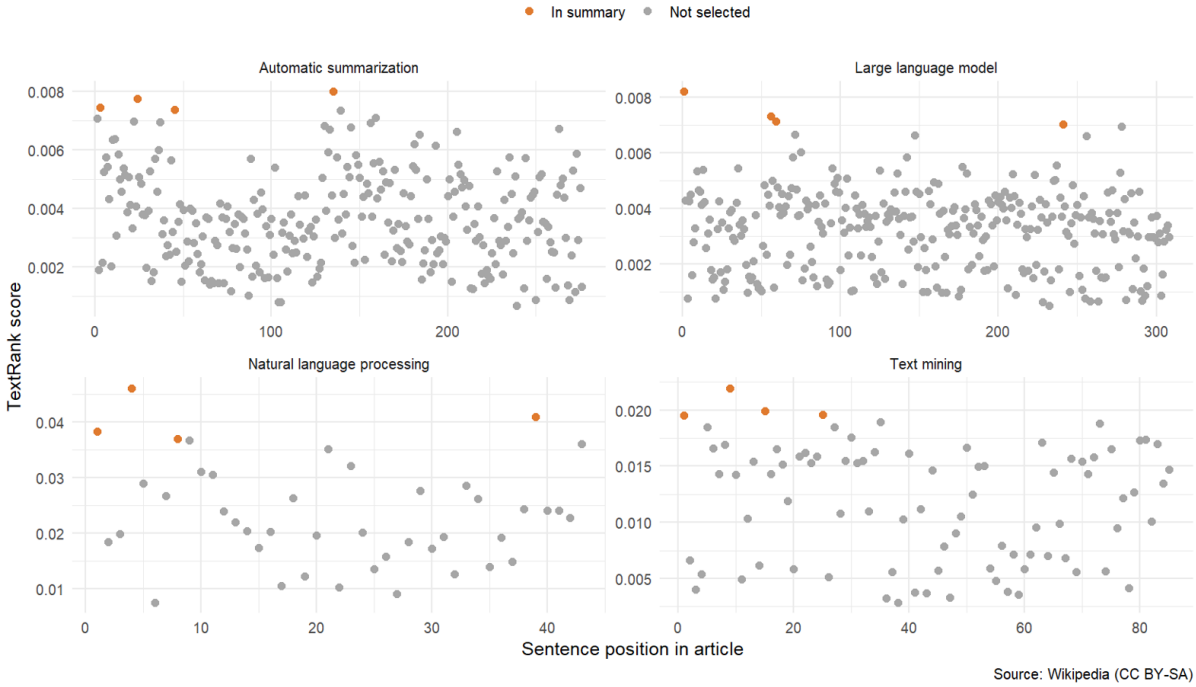


Figure 4 visualize the TextRank score of every sentence. The orange points are the 4 sentences chosen for each summary.

```r
#Technique 2: frequency-based extractive summary
word_freq <- sentence_words %>% count(article, word, name = "freq")
freq_scores <- sentence_words %>%
  left_join(word_freq, by = c("article", "word")) %>%
  group_by(article, sentence_id) %>%
  summarise(freq_score = mean(freq), .groups = "drop")

freq_summary <- sentences %>%
  inner_join(freq_scores, by = c("article", "sentence_id")) %>%
  group_by(article) %>% slice_max(freq_score, n = n_summary, with_ties = FALSE) %>%
  arrange(article, sentence_id) %>% ungroup()

#Technique 3: Text rank extractive summary
textrank_article <- function(a) {
  sw   <- sentence_words %>% filter(article == a) %>% distinct(sentence_id, word)
  ids  <- sort(unique(sw$sentence_id))
  sets <- split(sw$word, sw$sentence_id)
  pairs <- t(combn(ids, 2))
  w <- apply(pairs, 1, function(p) {
    s1 <- sets[[as.character(p[1])]]; s2 <- sets[[as.character(p[2])]]
    length(intersect(s1, s2)) / (log(length(s1) + 1) + log(length(s2) + 1))
  })
  edges <- data.frame(from = pairs[, 1], to = pairs[, 2], weight = w) %>% filter(weight > 0)
  g <- graph_from_data_frame(edges, directed = FALSE, vertices = data.frame(name = ids))
  data.frame(article = a, sentence_id = ids,
             textrank = as.numeric(page_rank(g, weights = E(g)$weight)$vector))
}
tr_scores <- bind_rows(lapply(unique(sentences$article), textrank_article))

textrank_summary <- sentences %>%
  inner_join(tr_scores, by = c("article", "sentence_id")) %>%
  group_by(article) %>% slice_max(textrank, n = n_summary, with_ties = FALSE) %>%
  arrange(article, sentence_id) %>% ungroup()


for (a in unique(sentences$article)) {
  cat("Frequency-based summary:\n")
  cat(paste0(" - ", freq_summary$sentence[freq_summary$article == a]), sep = "\n")
  cat("TextRank summary:\n")
  cat(paste0(" - ", textrank_summary$sentence[textrank_summary$article == a]), sep = "\n")
}

comparison <- sentences %>% group_by(article) %>%
  summarise(sentences_in_article = n(),
            words_in_article = sum(str_count(sentence, "\\S+"))) %>%
  left_join(textrank_summary %>% group_by(article) %>%
              summarise(words_in_summary = sum(str_count(sentence, "\\S+"))), by = "article") %>%
  left_join(inner_join(freq_summary, textrank_summary, by = c("article", "sentence_id")) %>%
              count(article, name = "sentences_chosen_by_both"), by = "article") %>%
  mutate(compression = sprintf("%.0f%%", 100 * words_in_summary / words_in_article),
         sentences_chosen_by_both = coalesce(sentences_chosen_by_both, 0L))
print(as.data.frame(comparison))


#  3.3
# Figure 1: most frequent words across all articles
f1 <- ggplot(top_words, aes(n, reorder(word, n))) +
  geom_col(fill = "#2f5d86") +
  geom_text(aes(label = n), hjust = -0.2, size = 3.5) +
  labs(title = "Figure 1: Top 15 words across the four scraped articles",
       subtitle = "After stop-word removal and lemmatisation",
       x = "Number of occurrences", y = "Word", caption = "Source: Wikipedia (CC BY-SA)") +
  theme_minimal(base_size = 12)
print(f1); ggsave("plots/1_top_words.png", f1, width = 8, height = 6, dpi = 150)

# Figure 2: words that characterise each article (TF-IDF)
f2 <- ggplot(tfidf, aes(tf_idf, reorder_within(word, tf_idf, article), fill = article)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ article, scales = "free") +
  scale_y_reordered() +
  theme_minimal(base_size = 12)
print(f2); ggsave("plots/2_tfidf_per_article.png", f2, width = 11, height = 7, dpi = 150)

# Figure 3: network of the most common word pairs
set.seed(2026)
bigram_graph <- graph_from_data_frame(head(bigrams, 40))
f3 <- ggraph(bigram_graph, layout = "fr") +
  geom_edge_link(aes(edge_width = n), edge_colour = "grey60", alpha = 0.7) +
  geom_node_point(colour = "#e07a2e", size = 3) +
  geom_node_text(aes(label = name), repel = TRUE, size = 3.5) +
  scale_edge_width(range = c(0.4, 2.5), name = "Times the pair occurs") +
  labs(title = "Figure 3: Network of the 40 most common word pairs (bigrams)",
       subtitle = "Linked words appear next to each other in the text",
       caption = "Source: Wikipedia (CC BY-SA)") +
  theme_void(base_size = 12)
print(f3); ggsave("plots/3_bigram_network.png", f3, width = 10, height = 7, dpi = 150)

# Figure 4: TextRank score of every sentence, with the selected summary sentences marked
tr_plot <- tr_scores %>%
  left_join(textrank_summary %>% transmute(article, sentence_id, selected = "In summary"),
            by = c("article", "sentence_id")) %>%
  mutate(selected = coalesce(selected, "Not selected"))
f4 <- ggplot(tr_plot, aes(sentence_id, textrank, colour = selected)) +
  geom_point(size = 2) +
  facet_wrap(~ article, scales = "free") +
  scale_colour_manual(values = c("In summary" = "#e07a2e", "Not selected" = "grey65")) +
  labs(title = "Figure 4: TextRank importance of each sentence",
       subtitle = paste("The", n_summary, "highest-scoring sentences per article form the summary"),
       x = "Sentence position in article", y = "TextRank score", colour = NULL,
       caption = "Source: Wikipedia (CC BY-SA)") +
  theme_minimal(base_size = 12) + theme(legend.position = "top")
print(f4); ggsave("plots/4_textrank_scores.png", f4, width = 11, height = 7, dpi = 150)
```

Question 4:

4.1) Using web scraping and regular expression (regex) analysis, define a use case of your own choosing. Develop an R program that scrapes at least 20 unstructured records (e.g., articles, news posts, or blog entries) from a website and constructs a data frame containing at least both a textual and a numerical (existing or inferred) attribute. The extracted data should initially contain spurious, inconsistent, or incomplete information representative of real-world web data

Use case:

Someone in cape town wants to know what type of events or activities there are around their location, under what category they fall and how much it will cost. No single site answers this. Different listing is spread across multiple sites. Each with their own format and unstructured text that hides dates, process and location.

The ETL pipeline turns this unstructured text into one relational table that can be queried by location, radius and category.

The R scraper collected 269 unstructured event records into one data frame. Each record has 6 attributes namely event name, description, venue, price, dates, start time.

The scraper followed fair use by following these practises:

- The scraper checks each site's robot.txt with paths_allowed() and will stop if the page is disallowed.
- An honest user agent is sent that names the project.
- The scraper has a 2 second waiting period between requests.
- Only reads public pages which does not require a login.
- Facts are kept and one-sentence summary rather than copying the whole article

Regex Extraction:

Attribute	Pattern (R stringr)	Example match
Labelled field	^Date:\s*(.*)$	Date: 13 November 2026 > 13 November 2026
Date and venue split	\p{So} (emoji symbol)	🗓️ 2 to 4 October 📍 Paarl > 2 to 4 October and Paarl
Month	(Jan|Feb|...|Dec)[a-z]*	September, Nov
Price in free text	R\s?\d[\d ,]*\d, (?i:free|gratis|no charge)	Tickets cost R150 per person > R150
Article links	(?<=<loc>)[^<]+ on the sitemap XML	.../whats-on-in-cape-town-in-may-2026/

Raw dataframe:

The raw dataframe has 269 rows and 11 columns (record_id, source, site_category, url, event_name, date_raw, time_raw, price_raw, venue_raw, description, published. The raw dataframe is saved as events_raw.csv.

(Code for 4.1)

```
library(rvest); library(robotstxt); library(dplyr); library(stringr)

if (rstudioapi::isAvailable()) setwd(dirname(rstudioapi::getSourceEditorContext()$path))
options(scipen = 999)

my_agent    <- "Student ETL project (R rvest, university assignment)"
pause_sec   <- 2
scrape_date <- Sys.Date()

sites <- c(woict = "https://whatsonincapetown.com",
           ctt   = "https://www.capetown.travel")

woict_categories <- c("live-events-cape-town", "performance", "markets-in-cape-town",
                      "food-and-drink-events", "kids-in-cape-town", "sports-events", "art-and-design",
                      "social-events-cape-town", "film-tech-gaming", "activities-experiences")
max_per_category <- 12



allowed <- paths_allowed(c(paste0(sites["woict"], "/event/"),
                           paste0(sites["ctt"], "/whats-on-in-cape-town-this-summer-2026/")))
if (!all(allowed)) stop("robots.txt does not allow one of the sites.")

get_page <- function(url) {
  Sys.sleep(pause_sec)
  tryCatch(session(url, httr::user_agent(my_agent)), error = function(e) NULL)
}
txt <- function(node) str_squish(html_text2(node))

month_pattern <- "(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)[a-z]*"
price_pattern <- "(?i:from\\s)?R\\s?\\d[\\d ,]*\\d|(?i:from\\s)?R\\s?\\d|(?i:\\bfree\\b|gratis|no charge)"

find_price <- function(text) {
  sapply(text, function(t) {
    s   <- unlist(str_split(t, "(?<=[.!?])\\s+|\n"))
    s   <- s[str_detect(s, regex("ticket|entry|entrance|admission|cost|price|per person|free", ignore_case = TRUE))]
    hit <- na.omit(str_extract(s, price_pattern))
    if (length(hit) == 0) NA_character_ else str_squish(hit[1])
  }, USE.NAMES = FALSE)
}

woict_hits <- bind_rows(lapply(woict_categories, function(cat) {
  pg <- get_page(paste0(sites["woict"], "/", cat, "/"))
  if (is.null(pg)) return(NULL)
  links <- unique(html_attr(html_elements(pg, "a[href*='/event/']"), "href"))
  links <- head(str_subset(links, "/event/[^/]+/$"), max_per_category)
  data.frame(site_category = rep(cat, length(links)), url = links)
}))

scrape_woict <- function(url) {
  pg <- get_page(url)
  if (is.null(pg)) return(NULL)
  meta  <- txt(html_elements(pg, ".event-meta p"))
  field <- function(label) {
    v <- na.omit(str_match(meta, paste0("^", label, ":\\s*(.*)$"))[, 2])
    if (length(v) == 0) NA_character_ else v[1]
  }
  data.frame(source = "What's On in Cape Town", url = url,
             event_name  = txt(html_element(pg, "h1")),
             date_raw    = field("Date"),
             time_raw    = field("Time"),
             price_raw   = field("Price"),
             venue_raw   = field("Venue address"),
             description = paste(html_text2(html_elements(pg, ".entry-content p")), collapse = "\n"))
}
woict <- woict_hits %>%
  left_join(bind_rows(lapply(unique(woict_hits$url), scrape_woict)), by = "url")

sitemap      <- httr::content(get_page(paste0(sites["ctt"], "/post-sitemap.xml"))$response,
                              as = "text", encoding = "UTF-8")
ctt_articles <- str_subset(str_extract_all(sitemap, "(?<=<loc>)[^<]+")[[1]], "whats-on-in-cape-town-.*2026")

scrape_ctt <- function(url) {
  pg <- get_page(url)
  if (is.null(pg)) return(NULL)
  paras <- html_elements(pg, ".tts_content_wrapper_1 p")
  ptxt  <- html_text2(paras)
  info  <- str_detect(ptxt, "\\p{So}") & str_detect(ptxt, month_pattern) & str_length(ptxt) < 200
  if (!any(info)) return(NULL)
  parts <- str_split_fixed(str_remove(ptxt[info], "^[\\s\\p{So}\\x{FE0F}]+"), "\\s*[\\p{So}\\x{FE0F}]+\\s*", 2)
  data.frame(source = "Cape Town Tourism", url = url,
             event_name  = sapply(paras[info], function(p)
               txt(html_element(p, xpath = "preceding-sibling::*[self::h2 or self::h3][1]"))),
             date_raw    = parts[, 1],
             venue_raw   = parts[, 2],
             description = sapply(paras[info], function(p)
               txt(html_element(p, xpath = "following-sibling::p[string-length(normalize-space(.)) > 40][1]"))),
             published   = html_attr(html_element(pg, "meta[property='article:published_time']"), "content"))
}
ctt <- bind_rows(lapply(ctt_articles, scrape_ctt)) %>%
  mutate(price_raw = find_price(description))

raw_df <- bind_rows(woict, ctt) %>%
  mutate(record_id = row_number(), .before = 1)

str(raw_df)
print(table(raw_df$source))
print(colSums(is.na(raw_df)))
write.csv(raw_df, "events_raw.csv", row.names = FALSE, fileEncoding = "UTF-8")
```

4.2) Apply appropriate data cleaning, denoising, and missing-value imputation techniques to improve the quality of the scraped dataset. Export the cleaned dataset as a CSV file suitable for further analysis

After cleaning the raw data, 212 clean events were captured. The clean dataframe is saved as events_clean.csv and places.csv with 48 places coordinates.

Step	What is does
Dates to start_date and end_date	Regex finds every "day month (year)", a missing year comes from the article date and rolls forward when it lands more than 60 days in the past
Recurring events	every, daily, weekly, weekends, monthly mark an event as Recurring, starting on the scrape date
Event type	Recurring, Ongoing if it runs for more than 31 days, otherwise Once-off
Start time to 24-hour HH:MM	7pm - 1.30am > 19:00, 15:00 - 15:30 > 15:00
Price to rand	The cheapest R amount; "free", "gratis" and "no charge" become R0
Spurious prices	Amounts under R10 or over R5 000 are not entry fees and are set to missing
Category	The site's own category where there is one, otherwise keyword regex on the name, then the description
Place and coordinates	A lookup of 48 suburbs and towns with venue and street aliases, matched in the venue, then the name, then the description
Denoise text	Drops captions (Image: ...), "Last updated", "Book now", Instagram embed text and lines under 6 words; converts curly quotes and accents to ASCII, keeps the first sentence as summary
Remove incomplete records	No event name, or no date that can be placed in time
Remove duplicates	Same normalised name (first 30 letters) and same start date: an event under two categories, or on both sites
Impute missing price	Median paid price of the event's category, else the median of all paid events, flagged in price_imputed
Impute missing location	Cape Town (general), the city's centre point, flagged in location_imputed
Export	write.csv(..., fileEncoding = "UTF-8", na = ""); empty cells load as NULL in PostgreSQL

(Code for 4.2)

```
month_num <- c(jan = 1, feb = 2, mar = 3, apr = 4, may = 5, jun = 6,
               jul = 7, aug = 8, sep = 9, oct = 10, nov = 11, dec = 12)

parse_dates <- function(x, ref) {
  if (is.na(x)) return(c(NA, NA))
  m <- str_match_all(x, regex(paste0("(\\d{1,2})(?:st|nd|rd|th)?\\s+", month_pattern,
                                     "\\.?(?:,?\\s+(\\d{4}))?"), ignore_case = TRUE))[[1]]
  if (nrow(m) == 0) return(c(NA, NA))
  day <- as.integer(m[, 2]); mon <- month_num[str_to_lower(substr(m[, 3], 1, 3))]
  yr  <- as.integer(m[, 4]); no_year <- is.na(yr)
  for (i in rev(seq_along(yr))) if (is.na(yr[i]) && i < length(yr)) yr[i] <- yr[i + 1]
  yr[is.na(yr)] <- as.integer(format(ref, "%Y"))
  d <- as.Date(sprintf("%d-%02d-%02d", yr, mon, day))
  d[no_year & d < ref - 60] <- d[no_year & d < ref - 60] + 365
  lead  <- as.integer(str_match(x, "^\\D*?(\\d{1,2})\\s*(?:to|-|\\u2013|&|and|,)\\s*\\d")[, 2])
  start <- if (!is.na(lead) && lead < day[1]) d[1] - (day[1] - lead) else d[1]
  c(start, max(d))
}

parse_time <- function(x) {
  m  <- str_match(x, regex("(\\d{1,2})(?:[:.h](\\d{2}))?\\s*(am|pm)?", ignore_case = TRUE))
  h  <- as.integer(m[, 2]); mi <- coalesce(as.integer(m[, 3]), 0L); ap <- str_to_lower(m[, 4])
  h  <- ifelse(!is.na(ap) & ap == "pm" & h < 12, h + 12L, ifelse(!is.na(ap) & ap == "am" & h == 12, 0L, h))
  ifelse(is.na(h) | h > 23 | mi > 59, NA, sprintf("%02d:%02d", h, mi))
}

parse_price <- function(x) {
  sapply(str_extract_all(x, "R\\s?\\d[\\d ,]*"), function(a) {
    v <- suppressWarnings(as.numeric(str_remove_all(a, "[R\\s,]")))
    v <- v[!is.na(v)]
    if (length(v) == 0) NA else min(v)
  })
}

category_rules <- c(
  "Markets"           = "market|bazaar|boeremark|craft fair",
  "Kids & Family"     = "kids|children|family|toddler|holiday programme",
  "Theatre & Comedy"  = "theatre|musical|comedy|stand-up|ballet|opera|magic|illusion|cabaret|drag|play\\b",
  "Music & Nightlife" = "concert|live music|dj\\b|band|jazz|party|club|album|gig|orchestra|choir|r&b|festival",
  "Food & Drink"      = "food|wine|beer|gin\\b|tasting|brunch|restaurant|dinner|lunch|breakfast|eatery|bistro|taproom|chef|menu",
  "Art & Culture"     = "art\\b|arts\\b|exhibition|gallery|museum|heritage|history|film|cinema|book",
  "Sport & Outdoors"  = "run\\b|marathon|race|cycl|hike|trail|rugby|cricket|soccer|surf|yoga|fitness|golf|swim|stormers",
  "Expos & Talks"     = "expo|summit|conference|talk|workshop|convention|show\\b")
classify <- function(text) {
  sapply(text, function(t) {
    hit <- names(category_rules)[str_detect(coalesce(t, ""), regex(paste0("\\b(", category_rules, ")"), ignore_case = TRUE))]
    if (length(hit) == 0) NA_character_ else hit[1]
  }, USE.NAMES = FALSE)
}
site_category_map <- c("live-events-cape-town" = "Music & Nightlife", "performance" = "Theatre & Comedy",
                       "markets-in-cape-town" = "Markets", "food-and-drink-events" = "Food & Drink",
                       "kids-in-cape-town" = "Kids & Family", "sports-events" = "Sport & Outdoors",
                       "art-and-design" = "Art & Culture", "film-tech-gaming" = "Art & Culture")
```

```
places <- read.csv(text = "place,pattern,latitude,longitude
V&A Waterfront,V&A|Waterfront|Zeitz MOCAA|Silo District|Robben Island Museum,-33.9036,18.4209
Cape Town CBD,City Centre|CBD|City Hall|Bree St|Long St|Loop St|Greenmarket|Company.?s Garden|Parade St,-33.9221,18.4231
Foreshore,Foreshore|CTICC|Convention Centre|Artscape,-33.9180,18.4300
District Six,District Six|Homecoming Centre|Harrington St,-33.9310,18.4290
Bo-Kaap,Bo-Kaap|Bo Kaap,-33.9200,18.4150
Newlands,Newlands|Kirstenbosch,-33.9800,18.4500
Milnerton,Milnerton|Montague Gardens,-33.8700,18.4950
Gardens,Gardens|Tamboerskloof|Oranjezicht|Vredehoek|Kloof St,-33.9330,18.4100
Green Point,Green Point|DHL Stadium|Cape Town Stadium,-33.9070,18.4060
Sea Point,Sea Point,-33.9150,18.3870
Camps Bay,Camps Bay|Clifton|Bantry Bay,-33.9510,18.3780
Hout Bay,Hout Bay,-34.0420,18.3520
Woodstock,Woodstock,-33.9280,18.4470
Salt River,Salt River,-33.9290,18.4630
Observatory,Observatory|Obz,-33.9380,18.4720
Rondebosch,Rondebosch|Baxter|UCT,-33.9630,18.4760
Claremont,Claremont|Kenilworth,-33.9840,18.4650
Wynberg,Wynberg,-34.0040,18.4660
Constantia,Constantia|Tokai,-34.0280,18.4220
Ottery,Ottery|Youngsfield,-34.0000,18.5160
Athlone,Athlone,-33.9620,18.5050
Langa,Langa,-33.9450,18.5300
Gugulethu,Gugulethu|Guguletu,-33.9800,18.5700
Khayelitsha,Khayelitsha,-34.0400,18.6700
Mitchells Plain,Mitchells Plain,-34.0500,18.6200
Muizenberg,Muizenberg|St James,-34.1080,18.4700
Kalk Bay,Kalk Bay,-34.1280,18.4490
Fish Hoek,Fish Hoek,-34.1360,18.4320
Simon's Town,Simon.?s ?Town|Boulders,-34.1920,18.4330
Noordhoek,Noordhoek|Kommetjie,-34.1000,18.1800
Century City,Century City|Canal Walk,-33.8920,18.5110
Table View,Table View|Blouberg|Bloubergstrand,-33.8240,18.4900
Goodwood,Goodwood|GrandWest|Grand West,-33.9120,18.5500
Bellville,Bellville|Tygerberg|Velodrome,-33.9000,18.6300
Durbanville,Durbanville,-33.8320,18.6470
Kuils River,Kuils River|Brackenfell,-33.9200,18.6900
Somerset West,Somerset West|Lourensford|Helderberg,-34.0840,18.8460
Strand,Strand(?!\\s+(St|Street))|Gordon.?s Bay,-34.1140,18.8270
Stellenbosch,Stellenbosch,-33.9321,18.8602
Franschhoek,Franschhoek|Boschendal|Pniel,-33.9133,19.1169
Paarl,Paarl,-33.7342,18.9621
Wellington,Wellington,-33.6390,19.0110
Philadelphia,Philadelphia|Durbanville Hills,-33.6670,18.5800
Darling,Darling(?!\\s+(and|St|Street))|Darling Brew,-33.3800,18.3800
Langebaan,Langebaan|West Coast,-33.0920,18.0280
Elgin,Elgin|Grabouw,-34.1500,19.0150
Hermanus,Hermanus|Overberg,-34.4187,19.2345
Cape Town (general),Cape Town,-33.9249,18.4241")
```

```
match_place <- function(text) {
  sapply(text, function(t) {
    if (is.na(t)) return(NA_character_)
    hit <- places$place[str_detect(t, regex(places$pattern, ignore_case = TRUE))]
    if (length(hit) == 0) NA_character_ else hit[1]
  }, USE.NAMES = FALSE)
}

clean_description <- function(x) {
  sapply(str_split(coalesce(x, ""), "\n"), function(lines) {
    lines <- str_squish(str_remove(lines, "Image:.*$"))
    lines <- lines[!str_detect(lines, regex("^(Last updated|View this post on Instagram|Book now)", ignore_case = TRUE))]
    paste(lines[str_count(lines, "\\S+") >= 6], collapse = " ")
  })
}
ascii <- function(x) str_squish(iconv(x, "UTF-8", "ASCII//TRANSLIT", sub = ""))

dates <- lapply(seq_len(nrow(raw_df)), function(i)
  parse_dates(raw_df$date_raw[i], coalesce(as.Date(substr(raw_df$published[i], 1, 10)), scrape_date)))

clean_df <- raw_df %>%
  mutate(
    start_date  = as.Date(sapply(dates, `[`, 1), origin = "1970-01-01"),
    end_date    = as.Date(sapply(dates, `[`, 2), origin = "1970-01-01"),
    recurring   = str_detect(coalesce(date_raw, ""), regex("every|daily|weekly|weekends?\\b|monthly|ongoing", ignore_case = TRUE)),
    start_date  = if_else(is.na(start_date) & recurring, scrape_date, start_date),
    event_type  = case_when(recurring ~ "Recurring",
                            as.numeric(end_date - start_date) > 31 ~ "Ongoing",
                            TRUE ~ "Once-off"),
    start_time  = parse_time(time_raw),
    price_raw   = coalesce(price_raw, find_price(description)),
    price_rand  = parse_price(price_raw),
    is_free     = is.na(price_rand) & str_detect(coalesce(price_raw, ""), regex("free|gratis|no charge", ignore_case = TRUE)),
    price_rand  = ifelse(is_free, 0, ifelse(between(price_rand, 10, 5000), price_rand, NA)),   # spurious prices
    category    = coalesce(unname(site_category_map[site_category]), classify(event_name), classify(description), "Other"),
    place       = coalesce(match_place(venue_raw), match_place(event_name), match_place(description)),
    event_name  = ascii(event_name),
    venue       = ascii(venue_raw),
    description = ascii(clean_description(description)),
    summary     = str_extract(description, "^.*?[.!?](?=\\s|$)")
  ) %>%
  filter(!is.na(event_name), !is.na(start_date)) %>%
  mutate(name_key = str_sub(str_remove_all(str_to_lower(event_name), "[^a-z0-9]"), 1, 30)) %>%
  distinct(name_key, start_date, .keep_all = TRUE)

clean_df <- clean_df %>%
  mutate(price_imputed    = is.na(price_rand),
         location_imputed = is.na(place),
         place            = coalesce(place, "Cape Town (general)")) %>%
  group_by(category) %>%
  mutate(price_rand = coalesce(price_rand, median(price_rand[price_rand > 0], na.rm = TRUE))) %>%
  ungroup() %>%
  mutate(price_rand = round(coalesce(price_rand, median(price_rand[price_rand > 0], na.rm = TRUE)))) %>%
  mutate(event_id = row_number()) %>%
  select(event_id, source, event_name, category, event_type, start_date, end_date, start_time,
         price_rand, is_free, price_imputed, venue, place, latitude, longitude, location_imputed,
         summary, url)

cat("Raw rows:", nrow(raw_df), "| Clean rows:", nrow(clean_df), "\n")
print(count(clean_df, source, category))
print(as.data.frame(head(select(clean_df, event_name, category, start_date, price_rand, place), 20)))

write.csv(clean_df, "events_clean.csv", row.names = FALSE, fileEncoding = "UTF-8", na = "")
write.csv(select(places, place, latitude, longitude), "places.csv", row.names = FALSE)
```

4.3) Using PostgreSQL, import the cleaned CSV dataset into a relational table. Perform two relevant SQL-based analytical investigations on the data. Present the SQL queries together with a concise interpretation of the results obtained.

```
\set my_place 'Cape Town CBD'
\set radius_km '0'
\set my_category 'All'

DROP TABLE IF EXISTS events;
DROP TABLE IF EXISTS places;

CREATE TABLE places (
    place VARCHAR(40) PRIMARY KEY,
    latitude NUMERIC(8, 5) NOT NULL,
    longitude NUMERIC(8, 5) NOT NULL
);

CREATE TABLE events (
    event_id INTEGER PRIMARY KEY,
    source VARCHAR(40) NOT NULL,
    event_name TEXT NOT NULL,
    category VARCHAR(30) NOT NULL,
    event_type  VARCHAR(15) NOT NULL,
    start_date DATE NOT NULL,
    end_date  DATE,
    start_time  TIME,
    price_rand  NUMERIC(8, 2),
    is_free BOOLEAN,
    price_imputed  BOOLEAN,
    venue  TEXT,
    place  VARCHAR(40) REFERENCES places (place),
    latitude  NUMERIC(8, 5),
    longitude  NUMERIC(8, 5),
    location_imputed BOOLEAN,
    summary  TEXT,
```

```
\copy places FROM 'C:/Users/ralle/Desktop/HNR/ITBDA4-14/Project 2/Question 4/places.csv' WITH (FORMAT csv, HEADER true)
\copy events FROM 'C:/Users/ralle/Desktop/HNR/ITBDA4-14/Project 2/Question 4/events_clean.csv' WITH (FORMAT csv, HEADER true, ENCODING 'UT

SELECT COUNT(*) AS events_loaded FROM events;

CREATE OR REPLACE FUNCTION distance_km(lat1 FLOAT, lon1 FLOAT, lat2 FLOAT, lon2 FLOAT)
RETURNS NUMERIC AS $$
    SELECT ROUND((6371 * 2 * ASIN(SQRT(
              POWER(SIN(RADIANS(lat2 - lat1) / 2), 2) +
              COS(RADIANS(lat1)) * COS(RADIANS(lat2)) * POWER(SIN(RADIANS(lon2 - lon1) / 2), 2)
          )))::NUMERIC, 1);
$$ LANGUAGE sql IMMUTABLE;

DROP TABLE IF EXISTS near_me;

CREATE TEMP TABLE near_me AS
SELECT e.*,
       distance_km(me.latitude, me.longitude, e.latitude, e.longitude) AS distance_km
FROM events e
CROSS JOIN (SELECT latitude, longitude FROM places WHERE place = :'my_place') AS me
WHERE (:radius_km = 0 OR distance_km(me.latitude, me.longitude, e.latitude, e.longitude) <= :radius_km)
  AND (:'my_category' = 'All' OR e.category = :'my_category');
```

Investigation 1: What is there to do and what will it cost?

```
SELECT category,
       COUNT(*) AS activities,
       SUM(CASE WHEN is_free THEN 1 ELSE 0 END)  AS free_activities,
       ROUND(100.0 * AVG(CASE WHEN is_free THEN 1 ELSE 0 END), 0) AS pct_free,
       ROUND(AVG(price_rand) FILTER (WHERE NOT price_imputed AND NOT is_free), 0) AS avg_paid_price,
       MIN(distance_km) AS nearest_km
FROM near_me
GROUP BY category
ORDER BY activities DESC;
```

```
     category      | activities | free_activities | pct_free | avg_paid_price | nearest_km
-------------------+------------+-----------------+----------+----------------+------------
 Music & Nightlife |         66 |               6 |        9 |            208 |        0.0
 Theatre & Comedy  |         34 |               1 |        3 |            225 |        0.0
 Food & Drink      |         32 |               1 |        3 |            502 |        0.0
 Art & Culture     |         25 |               6 |       24 |            138 |        0.0
 Markets           |         15 |               8 |       53 |                |        0.0
 Sport & Outdoors  |         15 |               1 |        7 |             60 |        0.0
 Expos & Talks     |         11 |               1 |        9 |            150 |        0.3
 Kids & Family     |          9 |               1 |       11 |            226 |        0.0
 Other             |          5 |               0 |        0 |                |        0.3
(9 rows)
```

Investigation 2: Display all the upcoming events around the chosen place:

```
SELECT place,
       ROUND(AVG(distance_km), 1)  AS distance_km,
       COUNT(*)  AS upcoming_activities,
       COUNT(DISTINCT category) AS categories,
       ROUND(AVG(price_rand), 0) AS avg_price,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS busiest_rank
FROM near_me
WHERE COALESCE(end_date, start_date) >= CURRENT_DATE OR event_type = 'Recurring'
GROUP BY place
ORDER BY upcoming_activities DESC, distance_km;
```

```
       place        | distance_km | upcoming_activities | categories | avg_price | busiest_rank
--------------------+-------------+---------------------+------------+-----------+--------------
 Cape Town (general) |         0.3 |                   8 |          4 |       229 |            1
 Cape Town CBD       |         0.0 |                   6 |          3 |       121 |            2
 Sea Point           |         3.4 |                   5 |          3 |       218 |            3
 Observatory         |         4.8 |                   5 |          3 |        34 |            3
 Foreshore           |         0.8 |                   4 |          2 |       191 |            5
 V&A Waterfront      |         2.1 |                   4 |          3 |       138 |            5
 Camps Bay           |         5.3 |                   3 |          3 |       187 |            7
 Newlands            |         6.9 |                   3 |          3 |       432 |            7
 Constantia          |        11.8 |                   3 |          1 |       108 |            7
 Gardens             |         1.7 |                   2 |          2 |       508 |           10
 Green Point         |         2.3 |                   2 |          2 |       255 |           10
 Century City        |         8.8 |                   2 |          2 |       100 |           10
 District Six        |         1.1 |                   1 |          1 |       180 |           13
 Rondebosch          |         6.7 |                   1 |          1 |        50 |           13
 Claremont           |         7.9 |                   1 |          1 |       400 |           13
 Ottery              |        12.2 |                   1 |          1 |       150 |           13
 Table View          |        12.5 |                   1 |          1 |         0 |           13
 Noordhoek           |        20.2 |                   1 |          1 |       180 |           13
 Stellenbosch        |        40.3 |                   1 |          1 |         0 |           13
 Somerset West       |        42.9 |                   1 |          1 |       495 |           13
 Franschhoek         |        64.0 |                   1 |          1 |         0 |           13
(21 rows)
```

The code contains a few other queries which I ran to play around. Feel free to have a look, I also built a UI to play around with the finding, but this was not in the project description and won't be included in the submission but will be included on the github repository.

4.4) Create a GitHub repository containing:

- **The project documentation (Questions 1-4)**
- **The project datasets**
- **The Notebooks showcasing the flow of analytics for all questions 1 to 4**

Andrew-le-Roux/ITBDA4-14_Project2: Big data analytics project: MapReduce, LDA topic modelling, web scraping and an R + PostgreSQL ETL pipeline